

## Chapter 15. Bias in Recommendation Systems

We've spent much time in this book dissecting how to improve our recommendations, making them more personalized and relevant to an individual user. Along the way, you've learned that latent relationships between users and user personas encode important information about shared preferences. Unfortunately, all of this has a serious downside: bias.

For the purposes of our discussion, we'll talk about the two most important kinds of bias for recommendation systems:

- Overly redundant or self-similar sets of recommendations
- Stereotypes learned by AI systems

First, we'll delve into the crucial element of diversity in recommendation outputs. As critical as it is for a recommendation system to offer relevant choices to users, ensuring a variety of recommendations is also essential. Diversity not only safeguards against overspecialization but also promotes novel and serendipitous discoveries, enriching the overall user experience.

The balance between relevance and diversity is delicate and can be tricky. This balance challenges the algorithm to go beyond merely echoing users' past behavior and encourages an exploration of new territories, hopefully providing a more holistically positive experience with the content.

This kind of bias is primarily a technical challenge; how do we satisfy the multiobjectives of diverse recommendations and highly relevant ones?

We'll consider the intrinsic and extrinsic biases in recommendation systems as an often unintended yet significant consequence of both the underlying algorithms and the data they learn from. Systemic biases in data collection or algorithmic design can result in prejudiced outputs, leading to ethical and fairness issues. Moreover, they may create echo chambers or filter bubbles, curtailing users' exposure to a broader range of content and inadvertently reinforcing preexisting beliefs.

At the end of this chapter, we will discuss the risks and provide resources to learn more about them. We are not experts in AI fairness and bias, but all ML practitioners should understand and seriously consider these topics. We aim to provide an introduction and signposts.

## Diversification of Recommendations

Our first investment into fighting bias is to explicitly target more diversity in our recommendation outputs. We'll briefly cover two of the many goals you may pursue: intra-list diversity and serendipitous recommendations.

*Intra-list diversity* attempts to ensure that there are a variety of types of items within a single recommendation list. The idea is to minimize similarity between the recommended items to reduce overspecialization and encourage exploration. High intra-list diversity within a set of recommendations increases the user's exposure to many items they may like; however, the recommendations for any particular interest will be shallower, reducing the recall.

*Serendipitous recommendations* are both surprising and interesting to the user. These are often items that the user might not have discovered independently or that are generally far less popular in the system. Serendipity can be introduced into the recommendation process by injecting nonobvious or unexpected choices—even if those have a relatively lower affinity score with the user—to improve overall serendipity. In an ideal world, these serendipitous choices are high affinity relative to other items of their popularity, so they're the “best of the outside choices.”

# Improving Diversity

Now that we have our measures of diversity, we can explicitly attempt to improve them. Importantly, by adding diversity metrics as one of our objectives, we will potentially sacrifice performance on things like recall or NDCG. It can be useful to think of this as a Pareto problem, or to impose a lower bound on ranking metric performance that you'll accept in pursuit of diversity.

---

## NOTE

In a *Pareto problem*, you have two priorities that often trade off with each other. In many areas of ML, and more generally applied mathematics, certain outcomes have a natural tension. Diversity in recommendations is an important example of a Pareto problem in recommendation systems, but it's not the only one. In [Chapter 14](#), you briefly saw global optimization, which is an extreme case of trade-offs.

---

One simple approach to improve diversity metrics is *reranking*: a post-processing step in which the initially retrieved recommendation list is re-ordered to enhance diversity. Various algorithms for re-ranking consider not just the relevance scores but also the dissimilarity among the items in the recommendation list. Re-ranking is a strategy that can operationalize any external loss function, so using it for diversity is a straightforward approach.

Another strategy is to break out of the closed loop of recommendation feedback that we discussed in the section [“Propensity Weighting for Recommendation System Evaluation”](#). As in multiarmed bandit problems, *explore-exploit trade-offs* can choose between exploiting what the model knows the user will like and exploring less certain options that may yield higher rewards. This trade-off can be used in recommendation systems to ensure diversity by occasionally choosing to *explore* and recommend less obvious choices. To implement a system like this, we can use affinity as a reward estimate and propensity as an exploitation measure.

Instead of using these posterior strategies, an alternative is to *incorporate diversity as an objective in the learning process* or include a diversity regularization term in the loss function. Multiobjective loss including pairwise similarity as a regularizer can help train the model to learn diverse sets of recommendations. You previously saw that kinds of regularization can coach the training process to minimize certain behaviors. One regularization term that can be used explicitly is *similarity among recommendations*; the dot product of each embedding vector in the recommendations to each other can approximate this self-similarity. Let  $\mathcal{R} = (\mathbf{R}_1, \mathbf{R}_2, \dots, \mathbf{R}_k)$  be the list of embeddings for the recommendations, and then consider  $\mathcal{R}$  as a column matrix—with each row a recommendation. Calculating  $\mathcal{R}$ 's Gramian would yield all our dot-product similarity calculations, and thus we can regularize by this term with appropriate hyperparameter weighting. Note that this differs from our previous Gramian regularization because we're considering the recommendations for only an individual query in this case.

Finally, we can use rankings from multiple domains to boost recommendation diversity. By integrating various ranking measures, the recommendation system can suggest items from outside the user's "mode," thus broadening the range of recommendations. A vibrant discipline exists around multimodal recommendations, with the [PinnerSage paper](#) from Pinterest a particularly impressive implementation. In many of the works about multimodal recommendations, the retrieval step returns too many recommendations near to the user's query vector. This forces self-similarity among the retrieved list. Multimodality forces multiple query vectors to be used for each request, allowing a built-in diversity.

Let's look at another perspective on item self-similarity and think about how the pairwise relationships between items can be used to this end.

## Applying Portfolio Optimization

*Portfolio optimization*, a concept borrowed from finance, can be an effective approach to enhance diversity in recommendation systems. The goal here is to create a "portfolio" of recommended items that balances the two key parameters: relevance and diversity.

At its heart, portfolio optimization is about risk (in our case, relevance) and return (diversity). Here's a basic approach for applying this optimization to recommendation systems:

1. Formulate an item representation such that the distance in the space is a good measure of similarity. This is in line with our previous discussions about what makes a good latent space.
2. Calculate pairwise distance between items. You can do this by using whatever distance metric that enriches your latent space. It is important to calculate these pairwise distances across all items retrieved and be ready for consideration to return. Note that how you aggregate these distributions of distances can be nuanced.
3. Evaluate affinity for the retrieved set. Note that calibrated affinity scores will perform better as they provide a more realistic estimate of return.
4. Solve the optimization problem. Solving the problem will yield a weight for each item that balances the trade-off between relevance and diversity. Items with higher weights are more valuable in terms of both diversity and relevance, and they should be prioritized in the recommendation list. Mathematically, the problem looks like this:

$$\text{Maximize}(w^T * r - \lambda * w^T * C * w)$$

Here,  $w$  is a vector representing the weights (i.e., the proportion of each item in the recommendation list),  $r$  is the relevance score vector,  $C$  is the covariance matrix (which captures the diversity), and  $\lambda$  is a parameter to balance relevance and diversity. The constraint here is that the sum of the weights equals 1.

Remember, the hyperparameter  $\lambda$  trades off between relevance and diversity. This makes it a critical part of this process and may require experimentation or tuning based on the specific needs of your system and its users. This would be straightforward via hyperparameter optimization in one of many packages such as Weights & Biases.

# Multiobjective Functions

Another related approach to diversity is to rank based on a multiobjective loss. Instead of the ranking stage being purely personalization affinity, introducing a second (or more!) ranking term can dramatically improve diversity.

The simplest approach here is something similar to what you learned in [Chapter 14](#): hard ranking. A business rule that may apply to diversity is limiting each item category to only one item. This is the simplest case of multiobjective ranking because sorting by a categorical column and selecting the max in each group will achieve explicit diversity with respect to that covariate. Let's move on to something more subtle.

In [“Stitching Together Spaces for Query-Based Recommendations”](#), one of this book's authors worked with coauthor Ian Horn to implement a multiobjective recommendation system that balanced both personalization and relevance to an image-retrieval problem.

The goal was to provide personalized recommendations for clothing that were similar to clothes in an image the user uploaded. This means there are two latent spaces:

- The latent space of personalized clothes to a user
- The latent space of images of clothing

To solve this problem, we first had to make a decision: what was more important for relevance? Personalization or image similarity? Because the product was centered around a photo-upload experience, we chose image similarity. However, we had another fact to consider: each uploaded image contained several pieces of clothing. As is popular in computer vision, we first segmented the model into several items and then treated each item as its own query (which we called *anchor-items*). This meant our image-similarity retrieval was multimodal, as we searched with several different query vectors. After we gathered them all, we had to make one final ranking—a multiobjective ranking for image similarity and personalization. The loss function we optimized is shown here:

$$s_i = \alpha \times (1 - d_i) + (1 - \alpha) \times a_i$$

The  $\alpha$  is a hyperparameter that represents the weighting,  $d_i$  is the image distance, and  $a_i$  is the personalization. We learn  $\alpha$  experimentally. The last step was to impose some hard ranking to ensure that one recommendation came from each anchor.

So let's sum this up:

1. We used two latent spaces with distances to provide rankings.
2. We did multimodal retrieval via image segmentation.
3. We retrieved using only one of the rankings.
4. Our final ranking was multiobjective, with hard ranking utilizing all our latent spaces and business logic.

This allowed our recommendations to be *diverse* in the sense that they achieved relevance in several areas of the query that corresponded to different items.

## Predicate Pushdown

You may be happy and comfortable applying these metrics during serving—after all, that's the title for this part of the book—but before we move on from this topic, we should talk about an edge case that can have quite disastrous consequences. When you impose the hard rules from [Chapter 14](#) and the diversity expectations discussed earlier in this chapter, and do a little multiobjective ranking, sometimes you arrive at...no recommendations.

Say you start by retrieving  $k$  items, but after the sufficiently diverse combinations that also satisfy business rules, there's simply nothing left. You might say, "I'll just retrieve more items; let's crank up  $k$ !" But this has some serious issues: it can really increase latency, depress match quality, and throw off your ranking model that is more tuned to lower-cardinality sets.

A common experience, especially with diversity, is that different modes for the retrieval have vastly different match scores. To take an example from our fashion recommender world: all jeans might be a better match than any shirt we have, but if you're looking for diverse categories of clothes to recommend, no matter how big the  $k$ , you'll potentially be missing out on shirts.

One solution to this problem is *predicate pushdown*. This optimization technique is used in databases, specifically in the context of data retrieval. The main idea of predicate pushdown is to filter data as early as possible in the data-retrieval process, to reduce the amount of data that needs to be processed later in the query execution plan.

For traditional databases, you see predicate pushdown applied, for example, as “apply my query’s `where` clause in the database to cut down on I/O.” It may achieve this by explicitly pulling the relevant columns to check the `where` clause first, and then getting the row IDs from those that pass before executing the rest of the query.

How does this help us in our case? The simple idea is if your vector store also has features for the vectors, you can include the feature comparisons as part of retrieval. Let's take an overly simple example: assume your items have a categorical feature called `color`, and for good diverse recommendations you want a nice set of at least three colors in your five recommendations. To achieve this, you can do a top- $k$  search across each of the colors in your store (the downside is that your retrieval is  $C$  times as large, where  $C$  is the number of colors that exist) and then do ranking and diversity on the union of these sets. This has a much higher likelihood of surviving your diversity rule in the eventual recommendations. This is great! We expect that latency is relatively low in retrieval, so this tax of extra retrievals isn't bad if we know where to look.

This optimization technique can be applied on quite complicated predicates if your vector store is set up well for the kinds of filters you wish to impose.

# Fairness

Fairness in ML in general is a particularly nuanced subject that is ill-served by short summaries. The following topics are important, and we invite you to consider the robust references included here:

## *Nudging*

Fairness does not need to be only “equal probabilities for all outcomes”; it can be fair with respect to a specific covariate. Nudging via a recommender—i.e., recommending items to emphasize certain behavior or buying patterns—can increase fairness. Consider the work by Karlijn Dinnissen and Christine Bauer from Spotify about [using nudging to improve gender representation in music recommendations](#).

## *Filter bubbles*

Filter bubbles are a downside of extreme collaborative filtering: a group of users begin liking similar recommendations, the system learns that they should receive similar recommendations, and the feedback loop perpetuates this. For a deep look into not only the concept but also mitigation strategies, consider [“Mitigating the Filter Bubble While Maintaining Relevance”](#) by Zhaolin Gao et al.

## *High risk*

Not all applications of AI are equal in risk. Some domains are particularly harmful when AI systems are poorly guardrailed. For a general overview of the most high-risk circumstances and mitigation, consult [Machine Learning for High-Risk Applications](#) by Patrick Hall et al. (O’Reilly).

## *Trustworthiness*

Explainable models is a popular mitigation strategy for risky applications of AI. While explainability does not *solve* the problem, it frequently provides a path toward identification and resolution. For a deep dive on this, [Practicing Trustworthy Machine Learning](#)

by Yada Pruksachatkun et al. (O'Reilly) provides tools and techniques.

### *Fairness in recommendations*

Because recommendation systems are so obviously susceptible to issues of AI fairness, much has been written on the topic. Each of the major social media giants has employed teams working in AI safety. One particular highlight is the Twitter Responsible AI team led by Rumman Chowdhury. You can read about the team's work in ["Can Auditing Eliminate Bias from Algorithms?"](#) by Alfred Ng.

## Summary

While these techniques provide pathways to enhance diversity, it's important to remember to strike a balance between diversity and relevance. The exact method or combination of methods used may vary depending on the specific use case, the available data, the intricacies of the user base, and the kind of feedback you're collecting. As you implement recommendation systems, think about which aspects are the most key in your diversity problem.